

基于深度学习的牛津宠物细粒度图像分类实验报告

姓名：蒋永胜 | 学号：W124301172 | GitHub: <https://github.com/WANGZAXIAOHONGGUAN/petproject>

1. 任务背景与实验设置

本实验在 Oxford-IIIT Pet 数据集上完成 37 类猫、狗品种的细粒度图像分类。与通用物体分类不同，细粒度分类的类间差异集中在耳形、面部花纹、毛色分布等局部区域，例如 British Shorthair 与 Russian Blue、American Pit Bull Terrier 与 Staffordshire Bull Terrier 在整体轮廓上高度相似，因此对模型判别性特征的提取能力要求更高。实验以 ImageNet 预训练的 ResNet-18 为骨干网络，仅将最后的全连接层替换为 37 类输出，全部训练与评估在单张 RTX 4060 Laptop (8 GB 显存) 上完成。

表 1 实验设置与超参数配置

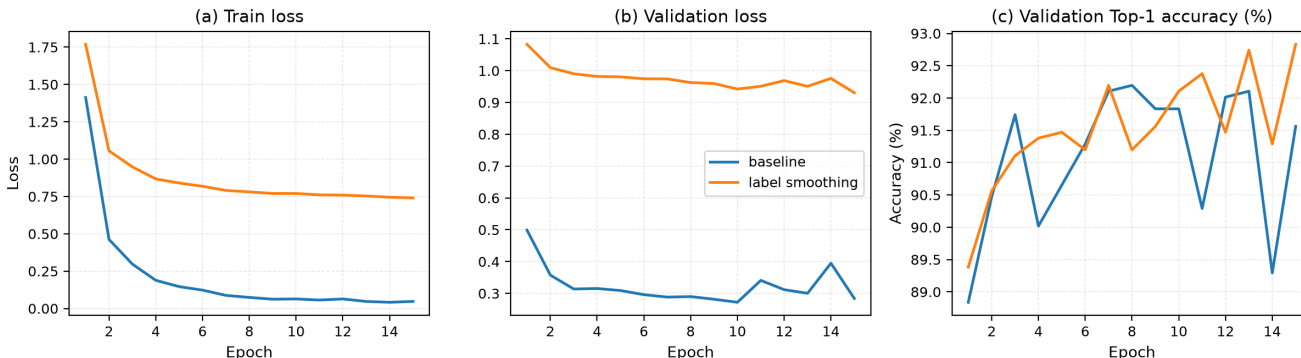
项目	配置
数据集	Oxford-IIIT Pet, 37 类, 合并官方 trainval 与 test 共 7349 张
数据划分	分层抽样 (Stratified) 70 : 15 : 15, seed=42 → 训练 5144 张 / 验证 1102 张 / 测试 1103 张
训练集预处理	Resize(256) + RandomCrop(224) + RandomHorizontalFlip + Normalize
验证/测试预处理	Resize(256) + CenterCrop(224) + Normalize (无任何随机增强)
骨干网络	ResNet-18 (ImageNet 预训练), fc 替换为 Linear(512, 37)
训练超参数	AdamW, lr=1e-4; Batch Size=32; 15 Epochs; CrossEntropyLoss
消融变量	损失函数: 标准 CrossEntropy → Label Smoothing CrossEntropy (0.1)
软硬件环境	Python 3.12 + PyTorch 2.14.0+cu126 + torchvision 0.29.0; RTX 4060 Laptop 8 GB / Ryzen 9 7945HX / Windows

2. 实验结果与消融分析

表 2 Baseline 与改进组的测试集指标对比 (留出测试集 1103 张)

实验配置	Top-1 Acc (%)	Top-5 Acc (%)	Macro-F1	最佳轮次	训练耗时
1. Baseline (ResNet-18 + 基础增强)	90.30	99.00	0.903	8 / 15	8.6 min
2. 改进组 (+ Label Smoothing 0.1)	92.38	98.55	0.923	15 / 15	8.6 min

图 1 训练过程曲线: 训练损失 / 验证损失 / 验证集 Top-1 准确率



两个模型都在 15 轮内完成收敛。Baseline 的训练损失由第 1 轮的 1.413 快速降到第 15 轮的 0.047，训练集 Top-1 从 70.6% 提升到 98.91%；但验证损失从第 3 轮起就在 0.27~0.40 之间徘徊（最后 8 轮均值 0.31），不再随训练损失同步下降，训练与验证精度之间始终存在约 7.4 个百分点的差距，说明模型已经明显过拟合。其验证准确率在第 8 轮达到峰值 92.20% 后开始回落，后 8 轮在 89.29%~92.20% 之间波动，因此最优权重按验证集准确率保存（Early-Stopping 式的模型选择）。

加入 Label Smoothing 后，训练损失的下限被抬高到约 0.75（这是软化标签下交叉熵的理论下限，并非收敛变差），训练与验证损失曲线之间的间距明显缩小，验证准确率则一直提升到第 15 轮仍未饱和，说明模型的泛化能力在持续改善。最终在测试集上，改进组的 Top-1 为 92.38%、Macro-F1 为 0.923，相比 Baseline 分别提升 2.09 个百分点与 0.021；Top-5 为 98.55%，与 Baseline 基本持平。这说明 Label Smoothing 通过抑制模型对训练样本的过度自信，缓解了过拟合，在不增加任何推理开销的前提下换来了更稳定的泛化表现。

3. 错误案例分析（Bad Case）与 Grad-CAM 可视化

为定位错误来源，我们在留出测试集上统计了 37×37 混淆矩阵（图 2 左，按行归一化为召回率），并把双向混淆样本数最多的类别对单独放大（图 2 右，单元格数字为样本数）。整体上混淆矩阵呈明显对角占优：37 个类别召回率的中位数为 92.9%，说明多数品种已被稳定区分；剩余错误高度集中在少数外观极相似的品种上（表 3）。

表 3 测试集上最容易混淆的类别对（双向样本数合并统计，前 4 对）

类别对	混淆样本数	混淆方向明细
Bengal 与 Egyptian Mau	8	Bengal → Egyptian Mau: 5 张; Egyptian Mau → Bengal: 3 张
American Pit Bull Terrier 与 Staffordshire Bull Terrier	7	American Pit Bull Terrier → Staffordshire Bull Terrier: 5 张; Staffordshire Bull Terrier → American Pit Bull Terrier: 2 张
British Shorthair 与 Russian Blue	7	British Shorthair → Russian Blue: 2 张; Russian Blue → British Shorthair: 5 张
Persian 与 Ragdoll	6	Persian → Ragdoll: 2 张; Ragdoll → Persian: 4 张

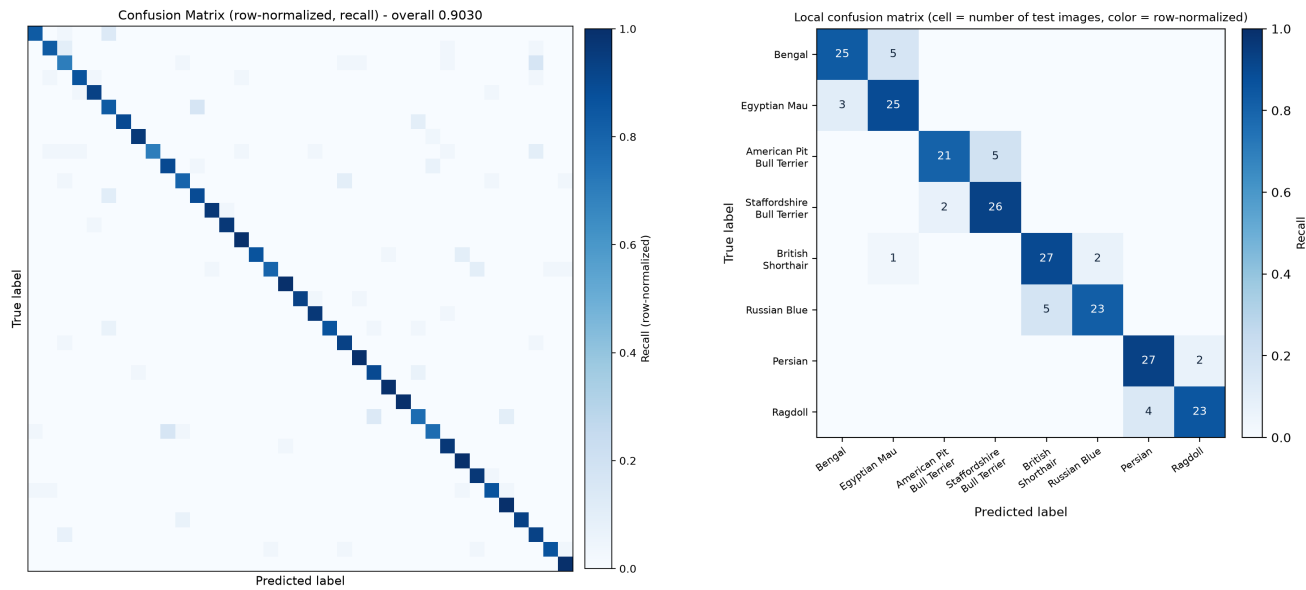


图 2 混淆矩阵：左为 37×37 整体（行归一化召回率），右为最易混淆类别构成的局部矩阵（单元格数字为该组合的测试样本数）

从表 3 可见，最严重的混淆是 Bengal 与 Egyptian Mau（8 张），其次为犬类 American Pit Bull Terrier 与 Staffordshire Bull Terrier（7 张）以及猫类 British Shorthair 与 Russian Blue（7 张）。这三对的共同点是整体体型与毛色斑纹高度相似，差异只体现在面部纹路、耳形等局部细节上。召回率最低的类别为 American Pit Bull Terrier（Recall=70.0%，F1=0.700）与 Boxer（Recall=70.0%），它们同时是拉低 Macro-F1 的主要来源。

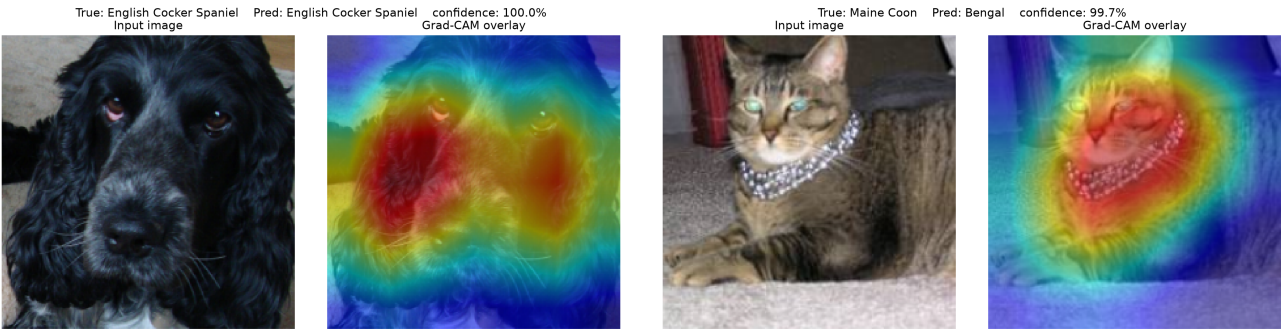


图 3 Grad-CAM 热力图（ResNet-18 的 layer4[-1]）：左为预测正确的样本（English Cocker Spaniel），右为预测错误的样本（真实 Maine Coon 误判为 Bengal），暖色表示网络决策时关注度更高的区域

预测正确的样本（English Cocker Spaniel，置信度 100.0%）中，Grad-CAM 的高响应区域集中在犬只的口鼻、眼部和耳部轮廓上，说明网络确实利用了与品种判别直接相关的面部结构特征；而预测错误的样本（真实为 Maine Coon，被误判为 Bengal，置信度 99.7%）中，高响应区域一方面落在额部的虎斑纹路上，另一方面明显落在与品种无关的颈部珠串项圈上——网络被大面积斑纹与配饰这类非判别性线索吸引，却没有利用缅因猫特有的耳簇毛与颈部围脖特征。在改进组模型上也能观察到同样倾向：Bengal 被误判为 Egyptian Mau（置信度 96.6%）时，热力图几乎全部落在身体斑点纹理上。这也与混淆矩阵的结论一致，即错误集中在斑纹、毛色相近的品种之间，属于细粒度分类中典型的“关键局部特征被背景与个体外观干扰”问题。

4. AI 辅助编程声明与 Bug 复盘

AI 使用情况：本次考核的环境配置命令、数据加载与分层划分（`data/dataset.py`）、训练循环（`train.py`）、混淆矩阵与 Grad-CAM 可视化脚本，均在与 AI 编程助手（ChatGPT / Codex）的多轮对话中生成，再由本人逐行阅读、运行、调试并修改；报告中的全部指标均由本人从 TensorBoard 日志与本地评估脚本输出中核对。

Bug 1（关键）：验证/测试集误用了训练集的随机增强，造成评估阶段的数据泄漏

- 现象：同一份权重在验证集上重复评估时结果不稳定，准确率会上下跳动若干个百分点；验证损失长期高于训练损失，与“过拟合”的解释不能完全对应。
- 定位：把 DataLoader 实际使用的变换打印出来，发现验证集的 transform 与训练集完全相同，其中包含 `RandomCrop` 和 `RandomHorizontalFlip`。根因是 AI 生成的代码在用 `Subset` 划分数据后，直接执行了 `train_dataset.dataset.transform = train_transform`：三个 `Subset` 共享同一个底层 Dataset 对象，这一行赋值把验证集与测试集的 transform 一并改掉了。这是典型的“共享对象副作用”错误，也恰好是任务书列出的头号暗坑。
- 修复：新增 `TransformSubset` 类，让训练 / 验证 / 测试三个 split 各自持有独立的 transform（训练集用 `RandomCrop + Flip`，验证与测试仅用 `CenterCrop`），并在 `dataset.py` 的自检入口打印三个 loader 的实际 transform 以便随时核查。
- 对照验证：用同一个 Baseline 权重做受控实验——测试集误用随机增强时，Top-1 为 89.45%（重复 5 次，波动 ± 0.42 个百分点，最低 88.85%）；修复为确定性 `CenterCrop` 后，Top-1 稳定在 90.30%，重复运行结果完全一致。可见该 Bug 不仅低估了模型精度，更让不同实验之间的对比失去了可比性。

Bug 2：数据划分不完整，且 Windows 控制台编码导致训练中断

- 第一版代码只用 `split='trainval'`（3680 张）进行划分，与任务书“加载全部约 7.3k 张”的要求不符；已修正为合并 `trainval` 与 `test` 后再做 70:15:15 分层抽样，得到 5144/1102/1103 的划分。
- 训练脚本用 `print` 输出 emoji 时，在 Windows 的 GBK 控制台抛出 `UnicodeEncodeError`，训练在第 1 轮结束后直接中断。改为纯文本输出并用 UTF-8 重定向日志后恢复正常，这也提醒我把“能跑起来”和“编码无关”分开验证。

体会：AI 生成的代码“语法正确”不等于“语义正确”。上述两个 Bug 都属于单看代码很难发现、却会实质性改变实验结论的问题（数据泄漏、划分与要求不一致）。可靠的做法是在关键路径上补自检断言——例如打印每个 split 实际使用的 transform、打印各子集样本数与标签取值范围——把“看起来跑通了”变成“可以证明是对的”。

5. 结论

在 7349 张、37 类的 Oxford-IIIT Pet 上，预训练 ResNet-18 + AdamW(lr=1e-4) + 15 轮即可取得 90.30% 的测试集 Top-1、99.00% 的 Top-5 与 0.903 的 Macro-F1，说明迁移学习是细粒度分类的高效基线；单变量消融表明 Label Smoothing (0.1) 能缩小训练与验证的差距，并把测试集 Top-1 提升到 92.38%（+2.09 个百分点）。混淆矩阵与 Grad-CAM 进一步表明，剩余错误集中在毛色斑纹相近的品种之间，且网络存在被配饰、背景纹理误导的倾向，因此后续改进的主要着力点是更强的数据增强（例如 MixUp、RandAugment）、局部区域注意力（如先定位头部再分类）以及更长的训练周期。